Open Loyalty Operating System (OLOS)

A vendor-neutral, privacy-preserving interoperability protocol for customer loyalty programs

---

Table of Contents

About

OLOS is an original open protocol proposal authored by Mark Angell. It defines a vendor-neutral, privacy-preserving interoperability framework for customer loyalty programmes, enabling secure issuance, redemption, settlement, and cross-network loyalty exchange between independent merchants, financial institutions, payment providers, and technology platforms.

This publication is intended to encourage industry discussion, technical evaluation, collaboration, and implementation across the global loyalty ecosystem.

Author	Mark Angell
Published by	OLOS Architecture Working Group
Version	1.0 (Executive Introduction) & 2.0.1 (Technical Specification)
Date	July 2026
Status	Public Proposal

Version History

Version	Date	Description
1.0	July 2026	First public release of the Executive Introduction.
2.0.0	July 2026	First public proposal of the modular Technical Specification.
2.0.1	July 2026	Errata pass: added the previously-referenced but unpublished OLOS-0009 (Escrow & Liquidity Management) module; completed Appendix A's Global Error Code Mapping Matrix; clarified OLOS-0010 §1 to resolve an apparent conflict between ingestion-time idempotency and clearing-time replay detection. No wire-level breaking changes from 2.0.0.

Preface

OLOS was created to address a longstanding structural limitation within the global loyalty industry: the absence of an open interoperability layer between independently operated loyalty programmes. While payment networks evolved through shared standards and settlement infrastructure, customer loyalty has largely remained fragmented into isolated proprietary ecosystems. OLOS proposes an open protocol that enables independent participants to exchange loyalty value while retaining full control over their own programmes, customer relationships, and commercial policies.

Part I — Executive Introduction

For CEOs, Merchants, Banks, Payment Networks & Investors

1. The Ledger No One Reconciles

The global loyalty industry operates on a trial balance that never closes. Every program keeps its own isolated books, and no single entity consolidates the balance. This has created three structural pain points across the industry:

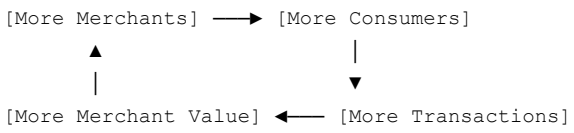
- **Liabilities —** Merchants collectively carry hundreds of billions of dollars in unredeemed points and miles liabilities on their balance sheets.
- **Holdings —** The average consumer holds dozens of separate loyalty balances, most of which are too small, narrow, or geographically isolated to ever be useful.
- **Redemption —** Redemption rates remain low because there is no common cross-network rail connecting one program's value to another's.

> **The Structural Reality:** This is not a marketing problem; it is a missing settlement layer. Payments achieved scale through shared network rails, and banking grew via data interchange standards. Customer loyalty has never had its equivalent — until now.

2. The Interoperability Loop

Unlike legacy coalitions, OLOS does not ask merchants to pool their programs into a single shared currency or surrender their margins. Instead, it introduces a common rail for sovereign programs.

This single architectural choice creates a powerful network flywheel:



Interoperability sits at the center of the loop. Every new participant compounds the utility of the entire mesh network, driving consumer engagement without degrading individual brand equity.

3. Mutual Ecosystem Value

Stakeholder	Core Strategic Benefit
Consumers	More places to redeem value, a simpler unified wallet experience, and highly relevant rewards.
Merchants	Higher customer engagement, lower integration/partnership costs, and instant ecosystem scalability.
Banks	Increased card usage, stronger merchant relationships, and new value-added financial services.
Loyalty Platforms	A significantly larger total addressable market with predictable, standards-based integrations.

4. High-Level Architecture Map

The system introduces two neutral infrastructure layers between market participants:

- **OLOS Gateway Layer** — Sits directly at the merchant point of sale (POS) to route, authorize, and validate loyalty transactions in flight.
- **OLOS Settlement & Netting Layer** — Reconciles netting matrices asynchronously between independent brands, creating the ledger that finally closes.

Part II — Technical Specification

Standards Edition

OLOS-0000: Architecture Overview

0. Requirement Terminology (RFC 2119 / RFC 8174)

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119, as clarified by RFC 8174, when, and only when, they appear in all capitals.

- **MUST** is reserved for requirements that directly affect wire-level interoperability between independently implemented nodes (envelope structure, signature verification, canonicalization, and event routing).
- **SHOULD** denotes strong guidance that a compliant implementation is expected to follow unless a documented, deliberate engineering tradeoff warrants deviation.
- **RECOMMENDED** is used specifically for default reference-implementation parameters (timeouts, cache TTLs, hardening postures).

1. Core Architectural Tenets

Every OLOS-compliant component MUST align with these four architectural pillars:

1. **Deterministic Integer-Only Math** — Floating-point numbers are strictly banned network-wide to eliminate cross-runtime rounding variance. All calculations are tracked as absolute integers scaled via exponents or basis points (1 bp = 0.0001).
2. **Structural Envelope Decoupling** — Message routers authenticate, validate, and route packets using an immutable, standardized outer envelope, allowing edge nodes to manage traffic without exposing internal payload data.
3. **Zero-Knowledge Identity Blinding** — Cleartext Personally Identifiable Information (PII) and raw primary account numbers (PANs) are barred from transmission or storage. Personas are blinded using localized, Hardware Security Module (HSM)-bound multi-party keys.
4. **Pure Asset Clearing** — The settlement layer operates strictly as a non-fiat asset netting mechanism, mapping obligations directly between participants.

2. High-Level System Topology

OLOS enforces complete decoupling between network trust governance and real-time transaction processing. The architecture isolates operations across five distinct logical planes:

Architectural Component	Core Responsibility	Logical Plane

Edge POS Gateway	Enforces structural envelope validations, detects payload tampering, and drops packets if temporal drift is greater than 5 minutes.	Ingestion / Data Plane
Trust Registry	Serves as the source of truth for node public keys, active merchant escrows, and FIPS hardware attestation manifests.	Governance / Control Plane
Rules Engine	Executes matrix criteria against transaction data inside isolated microVM environments.	Processing / Compute Plane
Identity Service	Derives unique Merchant-Specific Pseudonyms (MSPs) inside dedicated hardware enclaves using a high-entropy secret salt.	Privacy / Cryptographic Plane
Settlement Core	Aggregates issued and redeemed assets to compress complex multi-party debt loops into signed, immutable netting manifests.	Ledger / Clearing Plane

3. Core Structural Lifecycle

A standard transaction transitions through the system planes sequentially:

```
1. Ingestion  → Data Plane
2. Blinding   → Cryptographic Plane
3. Calculation → Compute Plane
4. Netting    → Clearing Plane
```

OLOS-0001: Message Envelope

Every payload transmitted across an OLOS network mesh MUST be enclosed within the standard root envelope. Gateways MUST drop any malformed message that omits any of the 8 mandatory root keys.

1. Envelope Field Specification

Field	Type	Multiplicity	Description
`olosVersion`	String	1..1	SemVer string indicating the protocol version (e.g., `"2.0.0"`).
`messageId`	String	1..1	Valid UUIDv4 generated by the producer for network-wide deduplication.
`correlationId`	String	1..1	Valid UUIDv4 distributed tracing tracer preserved across downstream paths.
`eventType`	String	1..1	Dot-notation namespaced classifier targeting specific event routers.
`timestamp`	String	1..1	ISO 8601 UTC timestamp format with millisecond precision.
`producerId`	String	1..1	Uniform Resource Name (URN) identifying the issuing node.
`signature`	String	1..1	Hex-encoded detached Ed25519 signature generated over canonical data.
`data`	Object	1..1	Functional payload container validated by target domain specifications.

2. Ingestion & Cryptographic Pipeline

When processing an envelope, an ingestion endpoint MUST execute the following sequence:

1. **Structural Sanity Check** — Verify payload size remains below 100 KB and all 8 root keys exist. Return `OLOS_ERR_MALFORMED_ENVELOPE` on failure.
2. **Replay Attack Mitigation** — Reject envelopes whose timestamp falls outside a bounded drift window relative to the gateway clock, returning `OLOS_ERR_REPLAY_WINDOW_EXCEEDED`. A drift tolerance of 5 minutes in the past and 5 seconds into the future is the RECOMMENDED default.
3. **Canonicalization** — Extract the inner data block and serialize it strictly using RFC 8785 JSON Canonicalization Scheme (JCS).
4. **Signature Verification** — Pull the public Ed25519 key assigned to `producerId` from the Trust Registry and verify it against the JCS bytes.

> **Note:** Step 2's drift check is a structural/transport-layer validation and is distinct from the messageId idempotency and long-term replay-index checks defined in OLOS-0010 §1 — a message can pass the drift window and still be rejected downstream as a duplicate or replay.

3. Envelope Reference Implementation

```
{
  "olosVersion": "2.0.0",
  "messageId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "correlationId": "a513da4c-112d-456a-86bb-7d884a1e99a2",
  "eventType": "protocol.tx.authorized",
  "timestamp": "2026-07-10T18:42:15.004Z",
  "producerId": "urn:olos:acquirer:acq_main_7701",
  "signature": "8e3cfa01b349d48b7f8c2de94132b982decd0234199c09bf87ad74ef1a293b118efdc03d922a1",
  "data": {
    "networkReference": "NET_REF_9921039810",
    "settlementClearingHouse": "urn:olos:settlement:settle_ch_01",
    "clearingMetrics": { "baseAmount": 4500, "currency": "USD", "exponent": 2 },
    "tokenizedIdentity": {
      "realm": "urn:olos:identity:zkp_provider_us",
      "proofToken": "zkp_tkn_88190239481"
    }
  }
}
```

Implementation Note: All numeric entries use an integer paired with an explicit base-10 exponent (e.g., `4500` with `exponent: 2` represents `45.00`). Floating points are entirely prohibited.

OLOS-0002: Event Registry

Network gateways and internal mesh fabrics MUST handle routing deterministically based on the envelope's structured `eventType` namespace.

1. Unified Event Namespace Tree

```
protocol (Root)
├── tx (Transaction Lifecycle Domain)
```

```
│   ├── authorized
│   ├── captured
│   ├── reversed   [OLOS-0003-EXT]
│   └── forwarded  [OLOS-0010 §3, offline recovery]
├── reward (Calculation Domain)
│   ├── issued
│   ├── redeemed
│   └── clawedback [OLOS-0003-EXT]
├── settlement (Netting Domain)
│   └── manifest.generated
├── escrow (Liquidity Domain)  [OLOS-0009]
│   ├── issued
│   ├── decremented
│   ├── throttled
│   └── replenished
├── identity (ZK Privacy Domain)
│   └── validated
└── fault (Diagnostics Domain)
    └── dlq
```

> **Correction:** The `protocol.tx.forwarded` cross-reference previously pointed to "OLOS-0010 §7," a section that does not exist in OLOS-0010 (which has three sections). It is corrected here to **OLOS-0010 §3**, the actual location of the Double-Envelope Wrapping Pattern. An `escrow` domain has also been added to the event tree, since OLOS-0009 (new in this revision) emits escrow lifecycle events that previously had no namespace home.

Unsupported extension events SHOULD route to `protocol.fault.dlq` with a capability-mismatch fault code rather than being treated as structurally malformed.

2. Dead-Letter Queue (DLQ) & Fault Isolation

If a message clears cryptographic and envelope validations but contains processing errors within its data sub-schema, it MUST NOT be silently dropped. The processing node MUST isolate the core envelope, excise the broken payload elements, attach an explicit error payload context mapping the fault code, and reroute the packet to `protocol.fault.dlq`.

OLOS-0003: Transaction Events

1. Lifecycle Constraints

```
POS Interaction ⟶ protocol.tx.authorized ⟶ protocol.tx.captured
```

This base module covers the active `authorized` and `captured` state steps. Reversal, void, chargeback, and clawback handling are defined separately in the OLOS-0003-EXT extension module. Base-profile nodes MUST NOT repurpose `protocol.tx.authorized` or `protocol.tx.captured` to represent a reversal.

2. Event Type: `protocol.tx.authorized`

Used for near-instantaneous, terminal-side accrual logic estimation or inline point redemptions.

```
{
  "transactionId": "tx_883910294",
```

```
  "terminalId": "term_pos_99218",
  "merchantId": "urn:olos:merchant:merch_992811",
  "transactionMetrics": {
    "authorizedAmount": 1480,
    "currency": "GBP",
    "exponent": 2
  },
  "paymentInstrument": {
    "type": "CARD_EMV",
    "maskedPan": "411111XXXXXX1111",
    "scheme": "VISA"
  },
  "identityToken": "id_tok_zknp_88291039"
}
```

3. Event Type: `protocol.tx.captured`

Represents the final, legally binding clearing record for a transaction.

```
{
  "captureId": "cap_11029384",
  "transactionId": "tx_883910294",
  "merchantId": "urn:olos:merchant:merch_992811",
  "transactionMetrics": {
    "authorizedAmount": 1480,
    "capturedAmount": 1750,
    "currency": "GBP",
    "exponent": 2
  },
  "clearingMetadata": {
    "batchId": "batch_20260710_04",
    "settlementDate": "2026-07-11"
  }
}
```

OLOS-0003-EXT: Transaction Reversals & Clawbacks

Status: OPTIONAL extension module. Nodes that have not declared this extension MUST treat reversal traffic as unsupported extension events.

This module provides a standardized data contract to mathematically undo accrual configurations without introducing ledger mutability.

1. Lifecycle Constraints

```
Point-of-Sale Intercept ⟶ protocol.tx.reversed ⟶ protocol.reward.clawedback
```

Immutability Rule — A reversal MUST NOT delete or alter historical records in the Settlement Core ledger. It instead issues a matching balancing vector processed in the current active Epoch domain.

2. Event Type: `protocol.tx.reversed`

```
{
```

```
  "reversalId": "rev_7710294c",
  "originalTransactionId": "tx_883910294",
  "merchantId": "urn:olos:merchant:merch_992811",
  "reversalMetrics": {
    "reversedAmount": 1750,
    "currency": "GBP",
    "exponent": 2
  },
  "reversalReason": "CUSTOMER_RETURN",
  "clawbackTarget": {
    "originalRewardId": "rwd_mint_9910283a",
    "allowNegativeBalance": true
  }
}
```

3. Clawback Calculation Formula

Engines evaluating a clawback action MUST reuse the identical modifier weights (`W_m,i`) and base multiplier (`M_base`) recorded in the original transaction's trace metadata.

4. Operational Invariants & Exception Handling

- **Negative Ledger Exemption —** If a clawback value is greater than the persona's current live balance, the system checks `allowNegativeBalance` . If `false` , the event MUST drop and route an `OLOS_ERR_CLAWBACK_NSF` alert to the DLQ.
- **Anti-Double-Reversal Lock —** Once accumulated `reversedAmount` against a unique transaction matches its absolute value, subsequent reversals MUST be rejected immediately with `OLOS_ERR_IDEMPOTENCY_VIOLATION` .

OLOS-0004: Rules Engine

The Rules Engine acts as a pure, stateless mathematical mapping function.

1. Mathematical Issuance Formula

Compliant engines MUST calculate reward distributions using integer arithmetic. The floor truncation function MUST occur exactly once, after the cumulative multiplication of all numerator parameters.

2. Equation Variable Dictionary

Variable	Description
`A_captured`	Raw integer value from the transaction metrics object.
`Δe`	Non-negative integer standardizing input scales (`e_target − e_tx`).
`M_base`	Core base conversion rate encoded directly in basis points (`10000 = 1.0×`).
`W_m,i`	Campaign, tier, or location modifiers encoded explicitly in basis points.

3. Event Type: `protocol.reward.issued`

```
{
```

```
  "rewardId": "rwd_mint_9910283a",
  "correlationId": "corr_88291039_abc",
  "identityToken": "id_tok_zknp_88291039",
  "merchantId": "urn:olos:merchant:merch_992811",
  "assetAllocation": {
    "assetType": "urn:olos:asset:points:coalition_miles",
    "amount": 148,
    "exponent": 0
  },
  "ruleTrace": {
    "matchedCampaigns": ["camp_summer_triple_2026", "camp_tier_gold_multiplier"],
    "executionNode": "urn:olos:network:node:rules_engine_eu_02"
  }
}
```

OLOS-0005: Settlement

The Settlement Engine calculates non-fiat loyalty asset obligations accumulated between network entities. It does not process or settle fiat currency.

1. Bilateral Liability Netting Principle

Net balances between any two participant nodes (A and B) are compressed into a single vector for an active Epoch time domain, where `O(A → B, k)` represents individual gross debit obligations originating from issuance or redemption configurations.

2. Event Type: `protocol.settlement.manifest.generated`

```
{
  "manifestId": "stle_man_20260710_01",
  "clearingCycle": {
    "epochId": "epoch_2026_week_28",
    "startTime": "2026-07-09T00:00:00.000Z",
    "endTime": "2026-07-09T23:59:59.000Z"
  },
  "nettingMatrix": [{
    "debtorMerchant": "urn:olos:merchant:merch_992811",
    "creditorMerchant": "urn:olos:merchant:merch_440192",
    "settlementMetrics": {
      "netTransferAmount": 894500,
      "currency": "EUR",
      "exponent": 2
    }
  }],
  "verificationStatus": "SETTLEMENT_SEALED"
}
```

3. Operational Invariants

- **Read-Only Ledger Lock** — Once a settlement manifest signature is written, affected historical engine balances are permanently frozen. Downstream adjustments must return `OLOS_ERR_IDEMPOTENCY_VIOLATION`.
- **Escrow-Bound Throttle** — If a merchant node's net liabilities exceed its escrow reserves in the Trust Registry, the gateway MUST drop outbound redemption paths and dispatch a high-severity alert, returning

`OLOS_ERR_ESCROW_EXCEEDED`. The reserve data model, sizing methodology, and full throttle lifecycle behind this invariant are normatively defined in **OLOS-0009**.

> **Correction:** The previous revision described this throttle narratively without naming a fault code or pointing to a data model. Both now exist in OLOS-0009.

OLOS-0006: Identity

To comply with global data protection frameworks (such as GDPR and CCPA), cleartext PII or raw transaction account tokens MUST NOT be logged, held, or shared.

1. Merchant-Specific Pseudonym (MSP) Derivation

The Identity Service decouples the static cross-network Payment Anchor from individual merchant domains using a keyed, multi-party blinding structure.

2. Variable Cryptographic Profile

Variable	Description
`P` (32-byte binary array)	The network-wide blinded payment baseline identifier, computed as `SHA-256(PAN)`.
`S` (32-byte high-entropy binary array)	A rotating cryptographic salt secret held exclusively within the Identity Service HSM enclave.
`M` (Variable-length UTF-8 string)	The explicit, namespaced `merchantId` string matching standard URN routing schemas.

Threat Model Boundary: Because `S` is locked within a hardware boundary and bound to the destination merchant's namespace `M`, cross-brand user profile reconstruction or transaction graph linkage is computationally infeasible under the stated threat model. This explicitly does **NOT** protect against a compromised Identity Service operator with access to raw salt `S`, or merchant-side cross-brand correlation performed using out-of-band identifiers (e.g., email captures at POS).

OLOS-0007: Governance & Incentives

Status: Draft — companion module addressing the non-technical layer the core specification assumes but does not define: who operates trust infrastructure, why participants join and stay, and how disputes get resolved.

1. Registry Operatorship

The Trust Registry (OLOS-0000 §2) cannot be operated unilaterally by any single commercial participant without recreating the closed-platform problem OLOS exists to solve. Three operating models are proposed for evaluation:

Model	Description	Tradeoff
Independent Foundation	A non-profit or industry consortium (modeled on EMVCo, W3C, or the Linux Foundation) holds registry keys and governs protocol changes.	Slower to stand up; highest neutrality and long-term trust.

Rotating Consortium Custody	Founding members (e.g., 5–9 seats across merchants, banks, and platforms) jointly operate the registry via threshold signatures (m-of-n).	Faster to launch; requires ongoing coordination among competitors.
Regulated Third Party	A neutral, licensed infrastructure provider (similar to a card scheme's role for authorization networks) operates the registry under contract and audit.	Introduces a paid intermediary; clearest accountability structure.

Recommendation for v1: Rotating Consortium Custody with a published path to Independent Foundation status once a critical mass of merchants (a defined threshold, e.g. 25+ independent nodes across 3+ verticals) has onboarded. This avoids a chicken-and-egg problem where no foundation exists to bootstrap, while committing publicly to decentralizing control.

Registry operators MUST NOT simultaneously act as a Settlement Core operator for the same network instance, to prevent a single entity from controlling both trust attestation and asset netting. **OLOS-0009** §3 **nds this same separation-of-duties requirement to escrow custody.**

2. Merchant Onboarding & Exit Rights

Onboarding

Merchants join at a declared Conformance Profile (OLOS-0011): Core Compliant, Enhanced Interoperable, or Enterprise Sovereign.

Onboarding requires: (a) HSM/TEE attestation registration, (b) an escrow deposit sized to expected redemption liability exposure per the methodology in **OLOS-0009 §3**, and (c) acceptance of a published Network Participation Agreement covering rate-setting participation, data handling, and dispute obligations.

No merchant is required to make its full points balance exchangeable — participants MAY designate a bounded percentage of issued points as network-exchangeable, preserving the sovereignty principle from the Executive Introduction.

Exit

- A merchant MAY exit at any time with a RECOMMENDED 90-day wind-down notice to allow outstanding netting obligations (OLOS-0005) to clear.
- On exit, unsettled obligations at the time of the final Epoch close MUST be honored against posted escrow before release (per OLOS-0009 §7) — a merchant cannot exit to avoid a settlement it already owes.
- Consumers holding a departing merchant's points SHOULD receive a published redemption grace window (RECOMMENDED: 180 days), communicated through the OLOS event bus, before that merchant's asset type is delisted from the network.

3. Rate-Setting & Economic Incentives

The core spec's Rules Engine (OLOS-0004) computes issuance mechanically, but the inputs — base multipliers and cross-merchant exchange weights — are economic decisions, not protocol decisions. These SHOULD be set through:

- A published, versioned rate schedule proposed by the registry operator(s) and subject to a comment/objection period before activation.
- Merchant-level override authority limited to their own outbound multiplier — a merchant can always make its own points more generous to redeem elsewhere, but cannot unilaterally reprice another merchant's asset.

Why merchants join: lower customer acquisition cost via shared network reach, reduced unredeemed-liability drag, and access to network-wide fraud/velocity signals (OLOS-0008/0010) they could not build alone.

Why merchants stay rather than defect to a closed model: exit forfeits the accumulated netting relationships and the escrow-based trust score built up over time — switching costs are intentionally asymmetric in favor of continued participation once integrated.

4. Dispute Resolution

Dispute Type	Resolution Path
Rate/valuation dispute (a merchant contests a settlement's exchange weight)	Escalates to a standing arbitration panel drawn from non-conflicted consortium members; frozen ledger state (OLOS-0005 §3) provides the immutable evidentiary record.
Clawback/reversal dispute (OLOS-0003-EXT)	Resolved algorithmically first via the deterministic clawback formula; only escalates to human arbitration if `allowNegativeBalance` and NSF conditions conflict with the merchant's stated policy.
Escrow/liquidity dispute (a merchant contests its required escrow sizing)	Routes to the same arbitration panel as rate/valuation disputes, per OLOS-0009 §6.
Registry misbehavior (a registry operator is accused of unauthorized key use or censorship)	Subject to the threshold-signature audit trail; a supermajority of remaining consortium seats MAY revoke and reissue that operator's registry authority.
Consumer-merchant redemption disputes	Explicitly out of scope for OLOS — these remain governed by the merchant's own existing terms of service, since OLOS mediates inter-merchant settlement, not consumer contracts.

All disputes that reach arbitration MUST be resolved using only data available from signed envelopes and sealed settlement manifests already on the network — no out-of-band evidence is admissible, keeping the dispute process consistent with the protocol's privacy and integrity guarantees.

5. Open Questions for Industry Comment

This module is deliberately left less rigid than the core protocol, since it depends on real commercial buy-in rather than pure engineering:

- ~~What escrow sizing formula fairly reflects a merchant's actual liability risk without over-collateralizing smaller participants?~~ **A reference formula is now proposed in OLOS-0009 §3**, though it remains open for refinement — see OLOS-0009's own open items.
- Should founding consortium seats be permanent or term-limited?
- Does registry operation require regulatory classification (e.g., as a payment system operator) in any target jurisdiction, and does that vary by which asset types are exchanged?

OLOS-0008: Security

1. Normative Transport Requirements (MUST)

- All inter-node network operations MUST use Mutual TLS (mTLS) over TLS 1.3. Legacy TLS 1.2 and unencrypted HTTP connections MUST be rejected at the edge.
- Nodes MUST support at least one of `TLS_AES_256_GCM_SHA384` or `TLS_CHACHA20_POLY1305_SHA256`, and MUST NOT negotiate a cipher suite outside this pair for OLOS traffic.
- Every cryptographic signing operation and MSP salt lookup MUST execute within an attested hardware isolation boundary (HSM or TEE) registered with the Trust Registry.

2. Reference Hardening Profile (RECOMMENDED — Non-Normative)

- Isolation boundary SHOULD meet FIPS 140-3 Level 3 certification parameters (e.g., AMD SEV-SNP or Intel TDX).
- Rules Engine compute blocks SHOULD execute inside hypervisor-isolated MicroVM environments. For multi-tenant deployments, isolating physical CPU cores per tenant and disabling Kernel Samepage Merging (`KSM=0`) are RECOMMENDED defenses.

OLOS-0009: Escrow & Liquidity Management

Status: Normative. *(New in v2.0.1 — this module is referenced as a mandatory dependency of the Enhanced Interoperable conformance profile in OLOS-0011 §1, and by OLOS-0005 §3 and OLOS-0007 §2, but was absent from v2.0.0. It is published here for the first time to close that gap.)*

Depends on: OLOS-0000 (Trust Registry), OLOS-0005 (Settlement), OLOS-0007 (Governance & Incentives)

1. Escrow Data Model

The Trust Registry (OLOS-0000 §2) MUST maintain an `EscrowAccount` record per registered merchant node:

```
{
  "escrowAccountId": "urn:olos:escrow:merch_992811",
  "merchantId": "urn:olos:merchant:merch_992811",
  "custodyModel": "REGISTRY_HELD",
  "reserveBalance": { "amount": 5000000, "currency": "USD", "exponent": 2 },
  "committedLiability": { "amount": 3120000, "currency": "USD", "exponent": 2 },
  "availableHeadroom": { "amount": 1880000, "currency": "USD", "exponent": 2 },
  "lastReplenishmentEpoch": "epoch_2026_week_27",
  "status": "ACTIVE"
}
```

`availableHeadroom` MUST always equal `reserveBalance − committedLiability` and MUST be recomputed atomically by the Settlement Core on every manifest generation (OLOS-0005 §2), not cached or estimated by edge nodes. See Appendix B.

2. Sizing Formula (Reference Methodology)

This module RECOMMENDS, rather than mandates, an initial sizing formula, since escrow sizing is an economic policy decision (OLOS-0007 §5) that individual registry operators MAY refine:

```
RequiredEscrow = PeakOfflineExposure + (TrailingRedemptionVelocity × ReplenishmentLatencyWindow)
```

Variable	Description

`PeakOfflineExposure`	Sum of all currently-issued offline escrow authorizations a merchant's edge devices could redeem while disconnected — i.e., total outstanding offline authorization ceiling across all its terminals.
`TrailingRedemptionVelocity`	A rolling average of net redemption obligations over a RECOMMENDED trailing 4-epoch window.
`ReplenishmentLatencyWindow`	The time between a reserve breach being detected and the merchant's next mandatory top-up, RECOMMENDED at 1 epoch.

Registry operators MAY apply a multiplier to `RequiredEscrow` (RECOMMENDED default: 1.15×) as a buffer against velocity spikes. Smaller participants SHOULD NOT be over-collateralized relative to actual issuance volume; sizing SHOULD scale with a merchant's trailing volume rather than a flat minimum.

3. Custody Models

Escrow custody follows the same operating-model choice as Registry Operatorship (OLOS-0007 §1):

Model	Description
`REGISTRY_HELD`	Reserve funds/asset backing held directly by the registry operator or its designated custodian.
`BONDED_THIRD_PARTY`	A regulated custodian holds reserves under a bond instrument; the registry only holds attestations of sufficient backing.
`SELF_ATTESTED_AUDITED`	Merchant retains custody of its own reserve, subject to periodic independent audit and real-time attestation feeds to the Trust Registry.

A registry operator MUST NOT simultaneously act as a Settlement Core operator (OLOS-0007 §1) — this module extends that same separation-of-duties requirement to escrow custody: an entity operating `REGISTRY_HELD` custody for a network instance MUST NOT also operate that instance's Settlement Core.

4. Throttle & Breach Handling

1. On every settlement manifest generation, the Settlement Core MUST recompute `availableHeadroom` for each participating merchant.
2. If a merchant's `committedLiability` would exceed `reserveBalance` as a result of processing a pending redemption, the gateway MUST reject that outbound redemption **before** it is committed, returning returning OLOS_ERR_ESCROW_EXCEEDED. See Appendix B.
3. A merchant whose `availableHeadroom` reaches zero MUST have its `EscrowAccount.status` transitioned to `THROTTLED`. While `THROTTLED`:
    - New offline escrow token issuance to that merchant's devices MUST be refused by the Trust Registry.
    - Previously issued, still-valid escrow tokens remain honorable up to their existing remaining amount — a throttle does not retroactively invalidate in-flight offline authorizations.
4. `status` MUST return to `ACTIVE` only after a replenishment transaction restores `availableHeadroom` above zero, confirmed by the Trust Registry.

This fault code has both a terminal-level and an account-level trigger, which implementations MUST distinguish in logging/telemetry even though they share a code:

Level	Checked by	Meaning
Terminal-level	Edge device, against its own local offline escrow allowance	A single device has exhausted its own offline authorization budget.
Account-level	Settlement Core, against the merchant's `EscrowAccount.availableHeadroom` (this module)	The merchant as a whole has insufficient reserve backing to cover its aggregate committed liability.

5. Fault Codes Introduced by This Module

Code	HTTP-equivalent Status	Trigger
`OLOS_ERR_ESCROW_EXCEEDED`	409	A redemption would push `committedLiability` above `reserveBalance` at the account level, or a terminal's local escrow allowance is exhausted at the device level.
`OLOS_ERR_ESCROW_REPLENISHMENT_OVERDUE`	409	An `EscrowAccount` remains `THROTTLED` past a RECOMMENDED grace period (default: 2 epochs) without replenishment, escalating from a throttle to a formal liquidity event subject to OLOS-0007 §4 dispute/arbitration handling.

Both codes are included in Appendix A.

6. Interaction with Governance (OLOS-0007)

- Escrow sizing disputes (a merchant contests its `RequiredEscrow` calculation) route to the same arbitration panel defined in OLOS-0007 §4 under "Rate/valuation dispute."
- A merchant that exits the network (OLOS-0007 §2) MUST have its escrow reserve held until all obligations at the final Epoch close are honored, consistent with the existing exit-rights language — this module formalizes the reserve mechanics behind that requirement.

7. Open Items

- The 1.15× sizing multiplier is a RECOMMENDED starting point, not derived from live network data, and should be revisited once real onboarding volume exists.
- This module intentionally does not mandate a single custody model as canonical for v1 certification — whether Enhanced Interoperable / Enterprise Sovereign profiles should require a specific model, or leave it operator-defined, is left open per OLOS-0007 §5's general posture on economic/governance questions.

OLOS-0010: Operational Resilience & Failure Semantics

1. Delivery & Retry Semantics

The transport layer observes at-least-once delivery. Consumers MUST treat `messageId` as an idempotency key. Retries MUST reuse the original `messageId` and `signature`.

This idempotency check operates in two tiers, which MUST NOT be collapsed into a single check:

1. **Ingestion-time idempotency (short-lived cache):** An ingestion node MUST first check `messageId` against a short-lived, edge-local delivery cache. A hit here means the same producer is resending an envelope whose original delivery attempt is still in flight or unacknowledged — ordinary at-least-once transport behavior. This case MUST be discarded without reprocessing, returning a **silent no-op**. It is not a fault and MUST NOT be routed to the DLQ.
2. **Clearing-time replay detection (long-term index):** Only if the short-lived cache misses does the node check `messageId` against the long-term `clearing_index` maintained by the Clearing Plane. A hit here means the transaction has already reached settlement finality (OLOS-0005 §3, "Read-Only Ledger Lock") in a prior epoch, and this is a resubmission of an already-settled economic event — regardless of whether the resubmission arrives in the original outer envelope or a new one (see OLOS-0010 §3 below). This case MUST be treated as `OLOS_ERR_REPLAY_ATTACK` and routed to `protocol.fault.dlq`.

> **Correction:** The v2.0.0 text of this section stated only that repeated `messageId`s "MUST be discarded without reprocessing, returning a silent no-op," without distinguishing pre-clearing retries from post-clearing replay. Read in isolation, this appeared to conflict with the replay-defense requirement that a duplicate `messageId` be faulted and routed to the DLQ. Both requirements are correct at their respective layer — the distinction above makes that explicit. Implementations MUST NOT silently no-op a `messageId` hit found only in the long-term index; doing so would suppress genuine replay/double-spend attempts from audit visibility.

2. Trust Registry Availability & Key Caching

Edge gateways SHOULD maintain a local, signed cache of active public keys with a bounded TTL (RECOMMENDED: 15 minutes). If the registry is unreachable, gateways MAY continue validating signatures using the cache for up to a 4-hour grace window before failing closed and returning `OLOS_ERR_TRUST_REGISTRY_UNAVAILABLE`.

3. Offline Recovery: The Double-Envelope Wrapping Pattern (Extension)

Status: OPTIONAL. This section describes an architectural pattern nodes MAY adopt to recover a stale offline envelope without weakening the global 5-minute replay-drift check in OLOS-0001.

Rather than relaxing the edge validation window, a reconnecting terminal wraps the stale message inside a fresh, compliant outer envelope under the `protocol.tx.forwarded` event type:


```
{
  "forwarderTerminalId": "term_pos_99218",
  "forwardedEpochId": "epoch_2026_week_28",
  "retrievalTracer": "fwd_tracer_8839102",
  "innerEnvelope": {
    "olosVersion": "2.0.0",
    "messageId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "correlationId": "a513da4c-112d-456a-86bb-7d884a1e99a2",
    "eventType": "protocol.tx.captured",
    "timestamp": "2026-07-10T12:00:00.000Z",
    "producerId": "urn:olos:acquirer:acq_main_7701",
    "signature": "8e3cfa01b349d48b7f8c2de94132b982decd0234199c09bf87ad74ef1a293b118efdc03d922",
    "data": {
      "captureId": "cap_11029384",
      "transactionId": "tx_883910294",
      "merchantId": "urn:olos:merchant:merch_992811",
      "transactionMetrics": { "authorizedAmount": 1480, "capturedAmount": 1750, "currency": "GBP", "expo
    }
```

```
    }
  }
```

Processing sequence for a forwarded envelope:

1. Outer Drift Check → Edge Plane
2. Outer Signature Check → Edge Plane
3. Inner Signature Check → Compute Plane
4. Long-Term Index Lookup → Clearing Plane (per §1 above)

OLOS-0011: Certification

1. Conformance Profiles Matrix

Profile Level	Mandatory Modules	Operational Objective	Scope / Target Environments
Core Compliant	OLOS-0001, 0002, 0003	Basic message ingestion, envelope sanity evaluation, and fault routing.	POS Edge Terminals, Acquirer Gateways.
Enhanced Interoperable	Core + OLOS-0004, 0005, 0009, 0010	Integer calculation processing, rule evaluation, obligation netting, escrow/liquidity management, and resilience semantics.	Coalition Hub Nodes, Regional Clearing Houses.
Enterprise Sovereign	Enhanced + OLOS-0006, 0007, 0008	Blinding identity processing, attested hardware isolation, and transport pinning enforcement.	Bank Core Infrastructure, Enterprise High-Risk Backbones.

> **Correction:** OLOS-0009 was already listed here as mandatory for Enhanced Interoperable in v2.0.0, but the module itself did not exist. It is now published above.

Appendix A: Global Error Code Mapping Matrix

Error Code	HTTP Status	Description Context	Corrective Mitigation Strategy
`OLOS_ERR_MALFORMED_ENVELOPE`	400	Missing required outer envelope parameters.	Drop packet immediately; do not forward to message bus.
`OLOS_ERR_REPLAY_WINDOW_EXCEEDED`	400	Envelope timestamp falls outside the permitted drift window.	Reject packet. Frequent occurrence from a single producer usually indicates terminal clock drift.
`OLOS_ERR_FLOATING_POINT_BANNED`	422	Non-integer financial or multiplier parameter detected.	Reject transaction; scale parameter to an integer paired with an exponent and retry.
`OLOS_ERR_INVALID_SIGNATURE`	401	Ed25519 validation fails against JCS canonical	Flag as potential tampering; log to audit vault; isolate network

		data bytes.	connection.
`OLOS_ERR_IDEMPOTENCY_VIOLATION`	409	Attempt to modify a sealed transaction or settlement manifest (e.g., a reversal exceeding the original transaction's absolute value).	Reject mutation; return historical state log to the client.
`OLOS_ERR_TRUST_REGISTRY_UNAVAILABLE`	503	Trust Registry unreachable and local key-cache grace window has expired.	Fail closed; route to `protocol.fault.dlq`; immediately alert network operations.
`OLOS_ERR_CLAWBACK_NSF`	409	Clawback amount exceeds the persona's live balance and `allowNegativeBalance` is `false` (OLOS-0003-EXT §4).	Reject clawback; do not apply partial reversal; escalate to merchant for manual reconciliation.
`OLOS_ERR_REPLAY_ATTACK`	409	Inner `messageId` already present in the long-term clearing index — distinct from an ingestion-cache idempotent no-op, which is not a fault (OLOS-0010 §1).	Reject; route to `protocol.fault.dlq`; flag for audit. Not to be confused with `OLOS_ERR_IDEMPOTENCY_VIOLATION`, which concerns post-seal mutation attempts rather than duplicate submission.
`OLOS_ERR_ESCROW_EXCEEDED`	409	Account-level reserve breach or terminal-level offline budget exhaustion (OLOS-0009 §4–5).	Reject the redemption pre-commit; transition `EscrowAccount.status` to `THROTTLED` if account-level.
`OLOS_ERR_ESCROW_REPLENISHMENT_OVERDUE`	409	`EscrowAccount` remains `THROTTLED` past the grace period without replenishment (OLOS-0009 §5).	Escalate to OLOS-0007 §4 arbitration path as a liquidity event.

> **Correction:** v2.0.0's Appendix A was missing four rows despite being titled the *global* error code matrix: `OLOS_ERR_CLAWBACK_NSF` and `OLOS_ERR_IDEMPOTENCY_VIOLATION` were normatively defined elsewhere in the spec but absent here; `OLOS_ERR_REPLAY_ATTACK` was implied by the replay-defense requirements but never formally listed; and `OLOS_ERR_ESCROW_EXCEEDED` / `OLOS_ERR_ESCROW_REPLENISHMENT_OVERDUE` are new alongside OLOS-0009.

```
<div align="center">
```

```
</div>
```

Appendix B: Headroom Verification Modes

Status: Normative. New in v2.0.2 — resolves an ambiguity between OLOS-0009 §1 and §4 identified during external technical review. Depends on: OLOS-0009 (Escrow & Liquidity Management), OLOS-0011 (Certification).

OLOS-0009 §1 states that availableHeadroom MUST be recomputed atomically by the Settlement Core on every manifest generation, implying batch-cycle computation. OLOS-0009 §4 separately requires a pre-commit check on every individual redemption, implying a real-time lookup. These are not the same operational model, and a node MUST declare which one it implements.

B.1 Verification Modes

- REAL_TIME — The gateway MUST query live availableHeadroom from the Settlement Core (or an authorized low-latency read replica) synchronously before committing each redemption. This is the strict reading of OLOS-0009 §4 step 2.
- BATCH_RECONCILED — availableHeadroom is authoritative only as of the most recent settlement manifest generation (OLOS-0009 §1). Redemptions are approved against that last-known value; any resulting over-commitment is detected retroactively at the next manifest cycle and handled as a liquidity event under OLOS-0009 §4–§5 (EscrowAccount.status transitions to THROTTLED, or OLOS_ERR_ESCROW_REPLENISHMENT_OVERDUE if unresolved), rather than blocked in real time.

B.2 Selection & Advertisement

Each EscrowAccount record (OLOS-0009 §1) MUST declare a headroomVerificationMode field with value REAL_TIME or BATCH_RECONCILED.

RECOMMENDED default: REAL_TIME for the Enterprise Sovereign conformance profile (OLOS-0011); BATCH_RECONCILED is acceptable for Enhanced Interoperable, reflecting its lighter infrastructure requirements.

A merchant switching modes SHOULD provide notice consistent with the OLOS-0007 §5 governance review process, since the change alters counterparties' risk exposure to that merchant's redemptions.

B.3 Interoperability Requirement

Because headroom verification mode determines whether a redemption can be provisionally trusted before settlement, nodes MUST treat a counterparty's advertised mode as binding. A node MUST NOT assume REAL_TIME guarantees when interacting with a merchant advertising BATCH_RECONCILED.

Appendix C: Capture vs. Authorization Amounts

Status: Non-normative clarification. New in v2.0.2. Clarifies OLOS-0003 §3.

The protocol.tx.captured reference example in OLOS-0003 §3 shows a capturedAmount (1750) exceeding the original authorizedAmount (1480). This is valid, not an error in the example.

capturedAmount MAY exceed authorizedAmount to account for tip adjustments or other post-authorization increases agreed at the point of capture, subject to any per-transaction ceiling enforced by the Rules Engine (OLOS-0004). Implementations MUST NOT reject a capture solely because capturedAmount > authorizedAmount.